VARDHAMAN
COLLEGE OF ENGINEERING

A Course End Project Report on

# ATM Simulation System

## A9513- Python Programming Laboratory


**Submitted By**

B.Pranay Chary
25881A05A7
B.Tanvi
25881A05C5

**Course Facilitator**

Ms P.S.Madhavi
Assistant Professor

**Department of Computer Science and Engineering,**

**Vardhaman College of Engineering, Hyderabad**

VARDHAMAN
COLLEGE OF ENGINEERING

Autonomous Institute, Affiliated to **JNTUH,**
Approved by **AICTE,** Accredited by **NAAC** with
**A**++ Grade Kacharam, Shamshabad, Hyderabad-
501 218, Telangana, India.

Vibrant Vardhaman
25 Years of

## DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

This is to certify that the Course End Project titled **"ATM Simulation System"** is carried out by **B.Pranay Chary** and **B.Tanvi** with Roll Numbers **25881A05A7 and 25881A05C5** towards **A9513 - Python Programming Laboratory** course in partial fulfilment of the requirements for the award of degree of **Bachelor of Technology** in **Computer Science and Engineering** during the Academic year 2025-26.

**Signature of the Course Faculty**
**Ms P.S.Madhavi**
**Assistant Professor, CSE**

**Signature** of the HOD

**Dr Y.VijayaLata,**
**Professor, Head of the Department & Dean(Student Mentoring)**
**Department of Computer Science and Engineering,**
**Vardhaman College of Engineering, Hyderabad,**

# Contents


1 Introduction ..... 3
2 Literature Review ..... 3
3 Objectives ..... 3
4 Methodology ..... 3
4.1 Architectural Diagram ..... 3
4.2 Hardware and Software Requirements ..... 3
4.3 Working Principle ..... 4
4.4 Software Implementation ..... 4
5 Results and Discussion ..... 4
6 Mapping POs and SDGs ..... 5
6.1 Program Outcomes (POs) Mapping ..... 5
6.2 Sustainable Development Goals (SDGs) Mapping ..... 5
6.3 Program Specific outcomes(PSOs) to Project Relavance ..... 5
7 Conclusion ..... 5
Bibliography ..... 6

# Abstract

The ATM Simulation System is a software-based application developed to simulate the essential operations of an Automated Teller Machine in a simple, secure, and user-friendly environment. The system is designed to provide users with basic banking services without requiring direct interaction with bank personnel. It focuses on implementing important ATM functionalities such as user authentication, balance enquiry, cash withdrawal, cash deposit, fund transfer, PIN change, transaction history, and secure exit from the system.

The application can be developed using Java and Object-Oriented Programming (OOP) concepts, with a graphical user interface to provide easy interaction between the user and the system. The user is first required to enter valid account credentials and a PIN. After successful authentication, the system displays a menu containing different banking operations. Each transaction is processed according to predefined rules, and the account balance is updated after every successful transaction. The system also performs input validation to handle incorrect PINs, invalid transaction amounts, insufficient account balance, and other inappropriate inputs.

The project demonstrates the practical application of programming concepts such as classes and objects, methods, encapsulation, conditional statements, loops, exception handling, and GUI programming. The system separates user interaction from transaction processing, making the application easier to understand, maintain, and extend. Transaction details can also be maintained to provide users with information about their previous operations.

The proposed ATM simulation provides an educational model of how automated banking systems process user requests and maintain account information. It helps students understand the relationship between authentication, transaction processing, data management, and user-interface design. The system can be further enhanced by integrating a database, stronger authentication mechanisms, encryption, OTP verification, receipt generation, and additional banking services.

Keywords: ATM Simulation, Automated Teller Machine, Java, Object-Oriented Programming, GUI, Authentication, Banking System, Transaction Processing, PIN Verification.

# 1. Introduction

An Automated Teller Machine (ATM) is an electronic banking system that enables customers to perform basic banking transactions without visiting a bank branch. It provides convenient services such as cash withdrawal, cash deposit, balance enquiry, PIN verification, and other account-related operations. ATM systems have become an important part of modern banking because they provide quick and convenient access to banking services.

The ATM Simulation System is a software-based application developed to demonstrate the basic working and operations of an ATM. The system allows users to log in using valid account details and a Personal Identification Number (PIN) before accessing banking services. After successful authentication, the user can select the required operation from the available menu. Each transaction is processed according to predefined conditions and the account balance is updated accordingly.

The system can be developed using Java programming and Object-Oriented Programming (OOP) concepts such as classes, objects, methods, encapsulation, conditional statements, loops, and exception handling. Input validation is used to handle incorrect PINs, invalid transaction amounts, and insufficient account balance. This helps the system provide appropriate responses and maintain accurate transaction processing.

The main purpose of this project is to provide a simple representation of how an ATM works while giving practical experience in developing a real-world software application. It helps in understanding authentication, transaction processing, user interaction, and basic security concepts. The system can be further enhanced by integrating a database, OTP verification, transaction history, receipt generation, and additional security features. Overall, the ATM Simulation System demonstrates how programming concepts can be combined to develop a simple, organized, and user-friendly banking application**.**

# 2. Literature Review

The development of automated banking systems has been widely studied as an application of computer-based transaction processing. Automated Teller Machines (ATMs) provide users with essential banking services such as cash withdrawal, balance enquiry, cash deposit, and account authentication. These systems are designed to reduce manual effort and provide users with convenient access to banking services.

Existing ATM systems generally focus on secure user authentication, transaction processing, and accurate management of account information. A PIN-based authentication mechanism is commonly used to verify the identity of a customer before allowing access to banking operations. Transaction validation is also important to prevent incorrect operations such as withdrawing an amount greater than the available balance.

Software-based ATM simulations are useful for understanding the basic concepts involved in real ATM systems. Such simulations allow students to implement banking operations using programming concepts such as classes, objects, methods, conditional statements, loops, and exception handling. Java is particularly suitable for developing such applications because its Object-Oriented Programming features support modular and structured software development.

Studies and existing implementations also show that ATM applications can be enhanced using databases and additional security mechanisms. Database integration can provide permanent storage of customer and transaction information, while features such as OTP verification, transaction history, encryption, and account locking can improve security. However, these advanced features may increase the complexity of the system.

The proposed ATM Simulation System focuses on providing a simple and user-friendly representation of basic ATM operations. It combines authentication, transaction processing, input validation, and account management in a single application. The project provides a practical understanding of how programming concepts can be applied to develop a real-world banking application and also provides a foundation for adding advanced banking and security features in future development.

# 3. Objectives

• **To implement and validate different banking operations such as cash withdrawal, cash deposit, balance enquiry, and PIN verification.**

This objective involves implementing essential ATM operations and validating each transaction according to predefined conditions. The system checks user authentication before providing access to banking services and verifies the entered transaction details before processing them. For example, during cash withdrawal, the system checks whether the requested amount is valid and whether sufficient balance is available. Similarly, deposits and balance enquiries are processed accurately, with the account balance updated after successful transactions. By providing appropriate messages for successful and unsuccessful operations, the system helps demonstrate secure and reliable transaction processing.

• **To provide secure user authentication and access control for ATM transactions.**

The system aims to authenticate users by verifying their account details and Personal Identification Number (PIN) before allowing access to banking operations. Invalid credentials are rejected to prevent unauthorized access to account information. The authentication process demonstrates the importance of basic security mechanisms in banking applications and ensures that transactions are performed only after successful verification.

• **To develop a simple and user-friendly interface for performing ATM operations.**

The system is designed to provide a clear and easy-to-use interface through which users can select the required banking service. Each operation is presented systematically so that users can easily understand the available options and receive appropriate messages after completing a transaction. The interface focuses on simplicity, clear navigation, and proper handling of user inputs**.**

• **To apply Java and Object-Oriented Programming concepts in developing the ATM system.**

The project aims to apply programming concepts such as classes, objects, methods, encapsulation, conditional statements, loops, and exception handling in a practical application. These concepts are

used to organize the different components of the ATM system and make the program structured, reusable, and easier to maintain.

- **To maintain accurate account information and ensure reliable transaction processing.**

The system is designed to maintain the user's account balance and update it whenever a valid transaction is completed. During withdrawals, the balance is reduced by the withdrawn amount, while during deposits, the balance is increased accordingly. The system also checks for invalid amounts and insufficient balance before processing transactions, helping to maintain consistency and accuracy.

- **To provide appropriate responses for successful and unsuccessful transactions.**

The system displays suitable messages based on the result of each operation. Successful transactions are confirmed to the user, while invalid inputs, incorrect PINs, insufficient balance, or other unsuccessful**.**

# 4. Methodology

## 4.1 Architectural Diagram

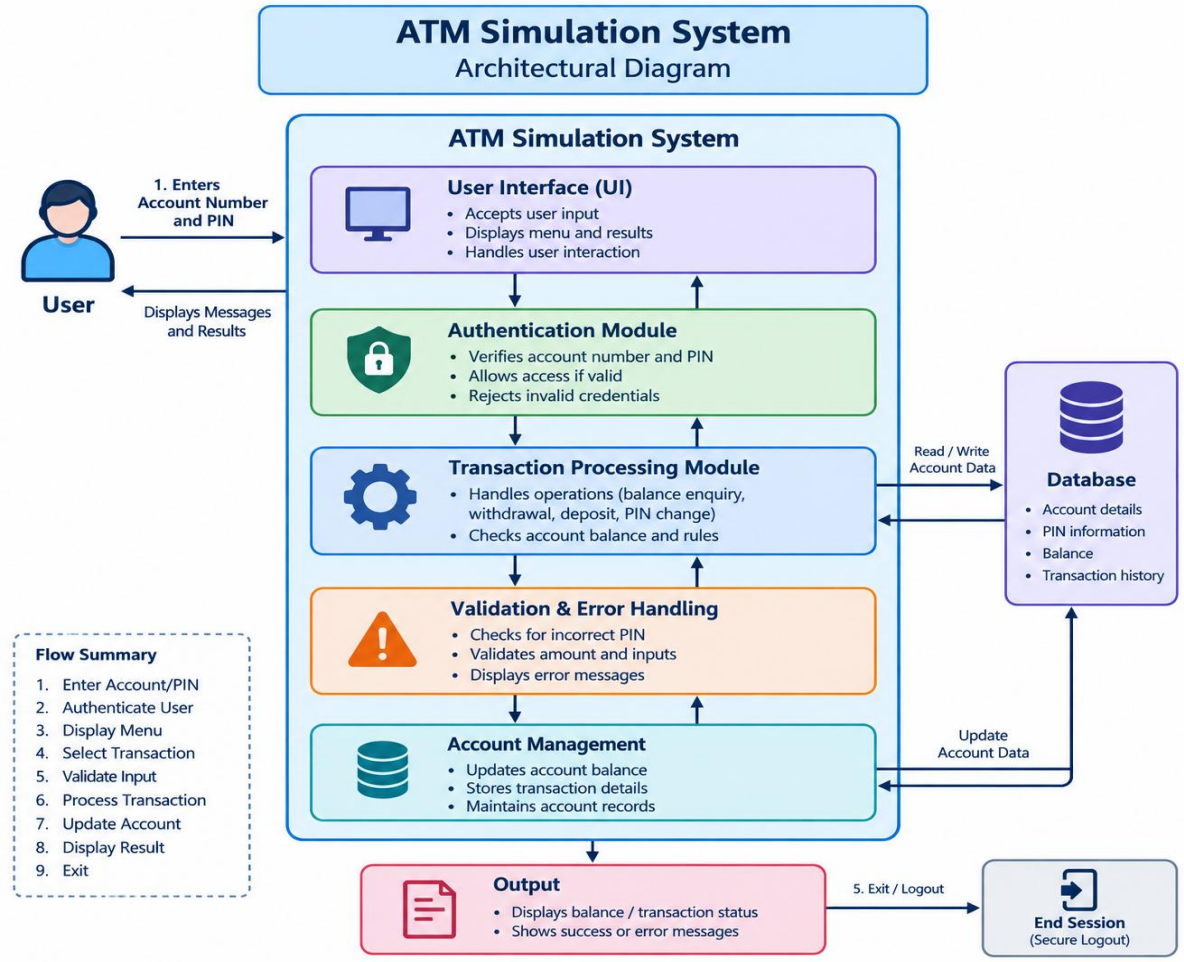



Figure 1: Architectural Diagram

The **ATM Simulation System** is designed to simulate the basic operations of a real Automated Teller Machine. The system begins by allowing the user to enter their account number and PIN. The entered details are verified to ensure that only authorized users can access the ATM services. After successful authentication, the user is provided with a menu of available banking operations.

The system provides various functions such as **balance enquiry, cash withdrawal, cash deposit, PIN change, and exit**. When the user selects an operation, the system processes the request according to the selected option. For example, during cash withdrawal, the system checks the available account balance before approving the transaction.Input validation and error handling are important parts of the system. The system checks for incorrect PINs, invalid amounts, insufficient balance, and invalid menu selections. Appropriate messages are displayed whenever an invalid input or unsuccessful transaction occurs. This helps make the system reliable, user.

## 4.2 Hardware and Software Requirements

Hardware Requirements

- Processor: Intel Core i3 or equivalent
- RAM: Minimum 4 GB
- Storage: Minimum 10 GB free space
- Display: 1366 × 768 resolution or higher
- Keyboard and Mouse: Required for user input
- Internet: Not required for basic ATM simulation

Software Requirements

- Operating System: Windows 10/11, Linux, or macOS
- Programming Language: Python
- IDE/Editor: VS Code, PyCharm, or IDLE
- Database: SQLite / MySQL
- Libraries: Tkinter for graphical user interface, SQLite3 for database connectivity
- Python Version: Python 3.x

## 4.3 Working Principle

The **ATM Simulation System** works by simulating the basic operations performed by a real ATM. First, the user enters their account number and PIN through the system interface. The system checks these details with the stored account information. If the credentials are correct, the user is granted access to the ATM menu; otherwise, an error message is displayed.

After successful authentication, the system displays different options such as **Balance Enquiry, Cash Withdrawal, Cash Deposit, PIN Change, and Exit**. The user selects the required operation, and the system accepts the necessary input. Each operation is processed according to the selected option.

Before completing a transaction, the system performs necessary **validation checks**. For example, during cash withdrawal, it checks whether the entered amount is valid and whether sufficient balance is available. If the conditions are satisfied, the transaction is completed; otherwise, an appropriate error message is displayed.

After a successful transaction, the system **updates the account information** in the database and displays the transaction result or updated balance. The user can perform another operation or select

Consider an **ATM Simulation System** with five customers performing different transactions. The system verifies the PIN, checks the account balance, processes the transaction, and updates the balance.

| Customer ID | Initial Balance (₹) | Transaction | Amount (₹) | Status |
|---|---|---|---|---|
| C1 | 10,000 | Withdraw | 2,000 | Successful |
| C2 | 5,000 | Deposit | 3,000 | Successful |
| C3 | 8,000 | Withdraw | 10,000 | Insufficient Balance |
| C4 | 12,000 | Withdraw | 5,000 | Successful |
| C5 | 6,000 | Deposit | 2,000 | Successful |

**ATM Processing Flow**

**C1:**

Enter PIN → PIN Verified → Select Withdrawal → Enter ₹2,000 → Balance Checked → Transaction Successful → **New Balance = ₹8,000**

**C2:**

Enter PIN → PIN Verified → Select Deposit → Enter ₹3,000 → Amount Added → Transaction Successful → **New Balance = ₹8,000**

**C3:**

Enter PIN → PIN Verified → Select Withdrawal → Enter ₹10,000 → Balance Checked → **Insufficient Balance** → **Transaction Failed**

**C4:**

Enter PIN → PIN Verified → Select Withdrawal → Enter ₹5,000 → Balance Checked → Transaction Successful → **New Balance = ₹7,000**

**C5:**

Enter PIN → PIN Verified → Select Deposit → Enter ₹2,000 → Amount Added → Transaction Successful → **New Balance = ₹8,000**

**Final Result**

| Customer | Initial Balance (₹) | Transaction (₹) | Final Balance (₹) | Result |
|---|---|---|---|---|
| C1 | 10,000 | Withdraw 2,000 | 8,000 | Successful |
| C2 | 5,000 | Deposit 3,000 | 8,000 | Successful |
| C3 | 8,000 | Withdraw 10,000 | 8,000 | Failed |
| C4 | 12,000 | Withdraw 5,000 | 7,000 | Successful |
| C5 | 6,000 | Deposit 2,000 | 8,000 | Successful |

**Working:**

**Enter Account/PIN → Authenticate → Select Transaction → Validate Amount → Process Transaction → Update Balance → Display Result → Logout**

## 4.4 Software Implementation

```
from datetime import date, datetime
import hashlib
import hmac
import json
from pathlib import Path
import secrets
import tkinter as tk
from tkinter import messagebox, ttk
import uuid


DAILY_WITHDRAWAL_LIMIT = 20000
PIN_HASH_ITERATIONS = 120000


class ATMApp:
    def __init__(self, root):
        self.root = root
        self.root.title("ATM Simulation System")
        self.root.geometry("600x760")
        self.root.minsize(540, 650)
        self.root.configure(bg="#f2f5f9")

        self.data_path = Path(__file__).with_name("atm_data.json")
        self.accounts = {}
        self.transactions = []
        self.cheque_numbers = set()
        self.account = None
        self.colors = {
            "navy": "#17324d",
            "blue": "#2364aa",
            "background": "#f2f5f9",
            "white": "#ffffff",
            "text": "#1f2937",
            "muted": "#64748b",
        }

        self.style = ttk.Style()
        self.style.configure("ATM.TButton", padding=(12, 8), font=("Segoe UI", 10))
        self.style.configure(
            "ATMPrimary.TButton",
            padding=(12, 9),
            font=("Segoe UI", 10, "bold"),
        )
        self.style.configure("ATM.TEntry", padding=8)

        if self.load_data():
            self.show_home()

    def load_data(self):
        if not self.data_path.exists():
            return True
        try:
```

```
            with self.data_path.open("r", encoding="utf-8") as data_file:
                data = json.load(data_file)
            self.accounts = data.get("accounts", {})
            self.transactions = data.get("transactions", [])
            if not isinstance(self.accounts, dict) or not isinstance(self.transactions,
list):
                raise ValueError("The saved ATM data has an unexpected format.")
            self.cheque_numbers = {
                item.get("cheque_number")
                for item in self.transactions
                if item.get("cheque_number")
            }
            return True
        except (OSError, json.JSONDecodeError, ValueError, AttributeError) as error:
            messagebox.showerror(
                "Could not load saved data",
                f"The ATM data file could not be read:\n{error}\n\n"
                "The app will close so the saved data is not overwritten.",
                parent=self.root,
            )
            self.root.destroy()
            return False

    def save_data(self):
        data = {"accounts": self.accounts, "transactions": self.transactions}
        temporary_path = self.data_path.with_suffix(".tmp")
        try:
            with temporary_path.open("w", encoding="utf-8") as data_file:
                json.dump(data, data_file, indent=2)
            temporary_path.replace(self.data_path)
            return True
        except OSError as error:
            messagebox.showerror(
                "Could not save data",
                f"The ATM data could not be saved:\n{error}",
                parent=self.root,
            )
            return False

    @staticmethod
    def make_pin_hash(pin, salt):
        return hashlib.pbkdf2_hmac(
            "sha256",
            pin.encode("utf-8"),
            bytes.fromhex(salt),
            PIN_HASH_ITERATIONS,
        ).hex()

    def pin_matches(self, account, pin):
        try:
            candidate = self.make_pin_hash(pin, account["pin_salt"])
            return hmac.compare_digest(candidate, account["pin_hash"])
        except (KeyError, ValueError):
            return False

    def clear_screen(self):
        for child in self.root.winfo_children():
            child.destroy()
```

```
def make_header(self, subtitle):
    header = tk.Frame(self.root, bg=self.colors["navy"], padx=28, pady=20)
    header.pack(fill="x")
    tk.Label(
        header,
        text="ATM Simulation System",
        font=("Segoe UI", 19, "bold"),
        fg="white",
        bg=self.colors["navy"],
    ).pack(anchor="w")
    tk.Label(
        header,
        text=subtitle,
        font=("Segoe UI", 10),
        fg="#d6e2f0",
        bg=self.colors["navy"],
    ).pack(anchor="w", pady=(5, 0))

def make_card(self):
    card = tk.Frame(self.root, bg=self.colors["white"], padx=28, pady=24)
    card.pack(fill="both", expand=True, padx=30, pady=24)
    return card

def add_field(self, parent, label, show=None):
    tk.Label(
        parent,
        text=label,
        font=("Segoe UI", 10, "bold"),
        fg=self.colors["text"],
        bg=self.colors["white"],
    ).pack(anchor="w", pady=(12, 5))
    entry = ttk.Entry(parent, style="ATM.TEntry", show=show or "")
    entry.pack(fill="x")
    return entry

def show_home(self):
    self.clear_screen()
    self.make_header("Create an account or log in to continue")
    card = self.make_card()
    tk.Label(
        card,
        text="Welcome",
        font=("Segoe UI", 18, "bold"),
        fg=self.colors["text"],
        bg=self.colors["white"],
    ).pack(anchor="w")
    tk.Label(
        card,
        text="Your accounts and transaction history are saved on this computer.",
        font=("Segoe UI", 10),
        fg=self.colors["muted"],
        bg=self.colors["white"],
        wraplength=450,
        justify="left",
    ).pack(anchor="w", pady=(6, 24))
    ttk.Button(
        card,
        text="Log in to an account",
        style="ATMPrimary.TButton",
```

```
            command=self.show_login,
        ).pack(fill="x", pady=5)
        ttk.Button(
            card,
            text="Create a new account",
            style="ATM.TButton",
            command=self.show_create_account,
        ).pack(fill="x", pady=5)
        ttk.Button(
            card,
            text="Close",
            style="ATM.TButton",
            command=self.root.destroy,
        ).pack(fill="x", pady=(18, 0))

    def show_create_account(self):
        self.clear_screen()
        self.make_header("Add another account to this ATM simulation")
        card = self.make_card()
        tk.Label(
            card,
            text="Create Account",
            font=("Segoe UI", 16, "bold"),
            fg=self.colors["text"],
            bg=self.colors["white"],
        ).pack(anchor="w")

        self.name_entry = self.add_field(card, "Account holder name")
        self.account_entry = self.add_field(card, "Account number (at least 6 digits)")
        self.pin_entry = self.add_field(card, "Create a 4-digit ATM PIN", show="*")
        self.balance_entry = self.add_field(card, "Opening balance (Rs.)")
        ttk.Button(
            card,
            text="Create account",
            style="ATMPrimary.TButton",
            command=self.create_account,
        ).pack(fill="x", pady=(22, 5))
        ttk.Button(card, text="Cancel", command=self.show_home).pack(fill="x", pady=4)

    def create_account(self):
        name = self.name_entry.get().strip()
        account_number = self.account_entry.get().strip()
        pin = self.pin_entry.get().strip()
        balance_text = self.balance_entry.get().strip()

        if not name or not all(part.isalpha() for part in name.split()):
            messagebox.showerror("Invalid name", "Enter a name using letters only.",
parent=self.root)
            return
        if not account_number.isdigit() or len(account_number) < 6:
            messagebox.showerror(
                "Invalid account number",
                "The account number must contain at least 6 digits.",
                parent=self.root,
            )
            return
        if account_number in self.accounts:
            messagebox.showerror(
                "Account already exists",
```

```
                "Choose a different account number.",
                parent=self.root,
            )
            return
        if len(pin) != 4 or not pin.isdigit():
            messagebox.showerror("Invalid PIN", "The PIN must contain exactly 4 dig-
its.", parent=self.root)
            return
        try:
            balance = float(balance_text)
        except ValueError:
            messagebox.showerror("Invalid balance", "Enter a valid opening balance.",
parent=self.root)
            return
        if balance < 0 or not balance.is_integer():
            messagebox.showerror(
                "Invalid balance",
                "The opening balance must be a non-negative whole amount.",
                parent=self.root,
            )
            return

        salt = secrets.token_hex(16)
        self.accounts[account_number] = {
            "name": name,
            "account_number": account_number,
            "pin_salt": salt,
            "pin_hash": self.make_pin_hash(pin, salt),
            "balance": balance,
            "failed_attempts": 0,
            "locked": False,
        }
        if self.save_data():
            messagebox.showinfo("Account created", "Your account was created success-
fully.", parent=self.root)
            self.show_login()

    def show_login(self):
        self.clear_screen()
        self.make_header("Enter your account number and PIN")
        card = self.make_card()
        tk.Label(
            card,
            text="Account Login",
            font=("Segoe UI", 16, "bold"),
            fg=self.colors["text"],
            bg=self.colors["white"],
        ).pack(anchor="w")
        self.login_account_entry = self.add_field(card, "Account number")
        self.login_pin_entry = self.add_field(card, "ATM PIN", show="*")
        ttk.Button(
            card,
            text="Log in",
            style="ATMPrimary.TButton",
            command=self.login,
        ).pack(fill="x", pady=(22, 5))
        ttk.Button(
            card,
            text="Create an account",
```

```
            command=self.show_create_account,
        ).pack(fill="x", pady=4)
        ttk.Button(card, text="Cancel", command=self.show_home).pack(fill="x", pady=4)

    def login(self):
        account_number = self.login_account_entry.get().strip()
        pin = self.login_pin_entry.get().strip()
        account = self.accounts.get(account_number)
        if account is None:
            messagebox.showerror("Login failed", "Account number was not found.", par-
ent=self.root)
            return
        if account.get("locked", False):
            messagebox.showerror(
                "Account locked",
                "This account is locked after three incorrect PIN attempts. Contact the
ATM administrator.",
                parent=self.root,
            )
            return
        if not self.pin_matches(account, pin):
            account["failed_attempts"] = account.get("failed_attempts", 0) + 1
            attempts_left = max(0, 3 - account["failed_attempts"])
            if attempts_left == 0:
                account["locked"] = True
                message = "Three incorrect PIN attempts. This account is now locked."
            else:
                message = f"Incorrect PIN. You have {attempts_left} attempt(s) remain-
ing."
            self.save_data()
            messagebox.showerror("Login failed", message, parent=self.root)
            return

        account["failed_attempts"] = 0
        if not self.save_data():
            return
        self.account = account
        self.show_atm_menu()

    def ask_fields(self, title, fields):
        dialog = tk.Toplevel(self.root)
        dialog.title(title)
        dialog.configure(bg=self.colors["white"])
        dialog.transient(self.root)
        dialog.resizable(False, False)
        dialog.grab_set()

        frame = tk.Frame(dialog, bg=self.colors["white"], padx=22, pady=18)
        frame.pack(fill="both", expand=True)
        entries = []
        for label, secret in fields:
            tk.Label(
                frame,
                text=label,
                font=("Segoe UI", 10, "bold"),
                fg=self.colors["text"],
                bg=self.colors["white"],
            ).pack(anchor="w", pady=(9, 4))
            entry = ttk.Entry(frame, style="ATM.TEntry", show="*" if secret else "")
```

```
            entry.pack(fill="x")
            entries.append(entry)

        result = {"values": None}

        def submit():
            result["values"] = [entry.get().strip() for entry in entries]
            dialog.destroy()

        def cancel():
            result["values"] = None
            dialog.destroy()

        buttons = tk.Frame(frame, bg=self.colors["white"])
        buttons.pack(fill="x", pady=(18, 0))
        ttk.Button(buttons, text="Submit", style="ATMPrimary.TButton", command=sub-
mit).pack(
            side="left", expand=True, fill="x", padx=(0, 5)
        )
        ttk.Button(buttons, text="Cancel", command=cancel).pack(
            side="left", expand=True, fill="x", padx=(5, 0)
        )
        dialog.protocol("WM_DELETE_WINDOW", cancel)
        if entries:
            entries[0].focus_set()
        dialog.bind("<Return>", lambda _event: submit())
        self.root.wait_window(dialog)
        return result["values"]

    @staticmethod
    def now_string():
        return datetime.now().astimezone().isoformat(timespec="seconds")

    @staticmethod
    def display_time(value):
        try:
            return datetime.fromisoformat(value).astimezone().strftime("%d-%m-%Y
%H:%M:%S")
        except (TypeError, ValueError):
            return str(value)

    def record_transaction(
        self,
        account_number,
        transaction_type,
        amount,
        balance,
        status="Completed",
        reference=None,
        cheque_number=None,
        recipient=None,
    ):
        self.transactions.append(
            {
                "date": self.now_string(),
                "account": account_number,
                "type": transaction_type,
                "amount": amount,
                "balance": balance,
```

```
                "status": status,
                "reference": reference or uuid.uuid4().hex[:10].upper(),
                "cheque_number": cheque_number,
                "recipient": recipient,
            }
        )
        return self.transactions[-1]

    def pending_cheque_total(self, account_number):
        return sum(
            float(item["amount"])
            for item in self.transactions
            if item.get("account") == account_number
            and item.get("type") == "Cheque Deposit"
            and item.get("status") == "Pending clearance"
        )

    def show_atm_menu(self):
        self.clear_screen()
        pending = self.pending_cheque_total(self.account["account_number"])
        self.make_header(f"Welcome, {self.account['name']}")
        card = self.make_card()
        tk.Label(
            card,
            text="ATM Main Menu",
            font=("Segoe UI", 16, "bold"),
            fg=self.colors["text"],
            bg=self.colors["white"],
        ).pack(anchor="w")
        tk.Label(
            card,
            text=(
                f"Account ending in {self.account['account_number'][-4:]}  •  "
                f"Available: Rs. {self.account['balance']:.2f}  •  "
                f"Pending cheques: Rs. {pending:.2f}"
            ),
            font=("Segoe UI", 9),
            fg=self.colors["muted"],
            bg=self.colors["white"],
            wraplength=470,
            justify="left",
        ).pack(anchor="w", pady=(5, 12))

        actions = [
            ("Balance enquiry", self.balance_enquiry),
            ("Cash deposit", self.deposit_money),
            ("Cheque deposit", self.deposit_cheque),
            ("Cash withdrawal", self.withdraw_money),
            ("Transfer to another account", self.transfer_money),
            ("Clear pending cheque deposits", self.clear_pending_cheques),
            ("Change ATM PIN", self.change_pin),
            ("Mini statement", self.mini_statement),
        ]
        for label, command in actions:
            ttk.Button(card, text=label, style="ATM.TButton", command=com-
mand).pack(fill="x", pady=2)
        ttk.Button(card, text="Log out", command=self.logout).pack(fill="x", pady=(10,
0))
```

```
    def refresh_menu(self):
        if self.account is not None:
            self.show_atm_menu()

    def parse_amount(self, value, label="amount"):
        try:
            amount = float(value)
        except ValueError:
            messagebox.showerror("Invalid amount", f"Enter a valid {label}.", par-
ent=self.root)
            return None
        if amount <= 0:
            messagebox.showerror("Invalid amount", "The amount must be greater than
zero.", parent=self.root)
            return None
        if not amount.is_integer():
            messagebox.showerror("Invalid amount", "Enter a whole amount in rupees.",
parent=self.root)
            return None
        return amount

    def show_receipt(self, title, lines):
        receipt = tk.Toplevel(self.root)
        receipt.title(title)
        receipt.geometry("450x470")
        receipt.minsize(390, 360)
        receipt.configure(bg=self.colors["white"])
        receipt.transient(self.root)

        tk.Label(
            receipt,
            text="ATM SIMULATION RECEIPT",
            font=("Segoe UI", 13, "bold"),
            fg=self.colors["navy"],
            bg=self.colors["white"],
        ).pack(anchor="w", padx=20, pady=(18, 8))
        text_area = tk.Text(
            receipt,
            wrap="word",
            height=18,
            font=("Consolas", 10),
            bg="#f8fafc",
            relief="flat",
            padx=12,
            pady=12,
        )
        text_area.pack(fill="both", expand=True, padx=20, pady=8)
        text_area.insert("1.0", "\n".join(lines))
        text_area.configure(state="disabled")
        ttk.Button(receipt, text="Close", command=receipt.destroy).pack(fill="x",
padx=20, pady=(4, 18))

    def receipt_lines(self, transaction):
        lines = [
            f"Date:      {self.display_time(transaction['date'])}",
            f"Type:      {transaction['type']}",
            f"Account:   {transaction['account']}",
            f"Amount:    Rs. {float(transaction['amount']):.2f}",
            f"Status:    {transaction.get('status', 'Completed')}",
```

```
            f"Reference: {transaction.get('reference', 'N/A')}",
        ]
        if transaction.get("recipient"):
            lines.append(f"Recipient: {transaction['recipient']}")
        if transaction.get("cheque_number"):
            lines.append(f"Cheque no: {transaction['cheque_number']}")
        lines.append(f"Balance:   Rs. {float(transaction['balance']):.2f}")
        return lines

    def balance_enquiry(self):
        pending = self.pending_cheque_total(self.account["account_number"])
        messagebox.showinfo(
            "Balance Enquiry",
            f"Account holder: {self.account['name']}\n"
            f"Account number: {self.account['account_number']}\n"
            f"Available balance: Rs. {self.account['balance']:.2f}\n"
            f"Pending cheque deposits: Rs. {pending:.2f}",
            parent=self.root,
        )

    def deposit_money(self):
        values = self.ask_fields("Cash Deposit", [("Deposit amount (Rs.)", False)])
        if values is None:
            return
        amount = self.parse_amount(values[0])
        if amount is None:
            return
        self.account["balance"] += amount
        transaction = self.record_transaction(
            self.account["account_number"], "Cash Deposit", amount, self.account["bal-
ance"]
        )
        if self.save_data():
            self.refresh_menu()
            self.show_receipt("Cash Deposit Receipt", self.receipt_lines(transaction))

    def deposit_cheque(self):
        values = self.ask_fields(
            "Cheque Deposit",
            [("Cheque number (6 to 12 digits)", False), ("Cheque amount (Rs.)", False)],
        )
        if values is None:
            return
        cheque_number = values[0]
        if not cheque_number.isdigit() or not 6 <= len(cheque_number) <= 12:
            messagebox.showerror(
                "Invalid cheque number",
                "The cheque number must contain 6 to 12 digits.",
                parent=self.root,
            )
            return
        if cheque_number in self.cheque_numbers:
            messagebox.showerror(
                "Cheque already deposited",
                "This cheque number has already been used.",
                parent=self.root,
            )
            return
        amount = self.parse_amount(values[1], "cheque amount")
```

```
        if amount is None:
            return

        self.cheque_numbers.add(cheque_number)
        transaction = self.record_transaction(
            self.account["account_number"],
            "Cheque Deposit",
            amount,
            self.account["balance"],
            status="Pending clearance",
            cheque_number=cheque_number,
        )
        if self.save_data():
            self.refresh_menu()
            lines = self.receipt_lines(transaction)
            lines.extend(["", "The amount is not available until the cheque is
cleared."])
            self.show_receipt("Cheque Deposit Receipt", lines)

    def daily_withdrawn(self, account_number):
        today = date.today()
        total = 0.0
        for item in self.transactions:
            if item.get("account") != account_number or item.get("type") != "Cash With-
drawal":
                continue
            try:
                if datetime.fromisoformat(item["date"]).astimezone().date() == today:
                    total += float(item["amount"])
            except (KeyError, TypeError, ValueError):
                continue
        return total

    def withdraw_money(self):
        values = self.ask_fields("Cash Withdrawal", [("Withdrawal amount (Rs.)",
False)])
        if values is None:
            return
        amount = self.parse_amount(values[0], "withdrawal amount")
        if amount is None:
            return
        if amount % 100 != 0:
            messagebox.showerror(
                "Invalid withdrawal",
                "Withdrawals must be in multiples of Rs. 100.",
                parent=self.root,
            )
            return
        remaining_limit = DAILY_WITHDRAWAL_LIMIT - self.daily_withdrawn(self.ac-
count["account_number"])
        if amount > remaining_limit:
            messagebox.showerror(
                "Daily limit exceeded",
                f"The daily withdrawal limit is Rs. {DAILY_WITHDRAWAL_LIMIT:.2f}.\n"
                f"You can withdraw up to Rs. {max(0, remaining_limit):.2f} more today.",
                parent=self.root,
            )
            return
        if amount > self.account["balance"]:
```

```
            messagebox.showerror(
                "Insufficient balance",
                "There is not enough money in the account.",
                parent=self.root,
            )
            return

        self.account["balance"] -= amount
        transaction = self.record_transaction(
            self.account["account_number"], "Cash Withdrawal", amount, self.ac-
count["balance"]
        )
        if self.save_data():
            self.refresh_menu()
            self.show_receipt("Withdrawal Receipt", self.receipt_lines(transaction))

    def transfer_money(self):
        values = self.ask_fields(
            "Transfer to Another Account",
            [("Recipient account number", False), ("Transfer amount (Rs.)", False)],
        )
        if values is None:
            return
        recipient_number = values[0]
        if not recipient_number.isdigit() or len(recipient_number) < 6:
            messagebox.showerror("Invalid account number", "Enter a valid account num-
ber.", parent=self.root)
            return
        if recipient_number == self.account["account_number"]:
            messagebox.showerror("Invalid transfer", "You cannot transfer to the same
account.", parent=self.root)
            return
        recipient = self.accounts.get(recipient_number)
        if recipient is None:
            messagebox.showerror("Account not found", "The recipient account does not
exist.", parent=self.root)
            return
        if recipient.get("locked", False):
            messagebox.showerror("Account locked", "The recipient account is locked.",
parent=self.root)
            return
        amount = self.parse_amount(values[1], "transfer amount")
        if amount is None:
            return
        if amount > self.account["balance"]:
            messagebox.showerror(
                "Insufficient balance",
                "There is not enough money in the account.",
                parent=self.root,
            )
            return

        reference = uuid.uuid4().hex[:10].upper()
        self.account["balance"] -= amount
        recipient["balance"] += amount
        transaction = self.record_transaction(
            self.account["account_number"],
            "Transfer Sent",
            amount,
```

```
            self.account["balance"],
            reference=reference,
            recipient=recipient_number,
        )
        self.record_transaction(
            recipient_number,
            "Transfer Received",
            amount,
            recipient["balance"],
            reference=reference,
            recipient=self.account["account_number"],
        )
        if self.save_data():
            self.refresh_menu()
            self.show_receipt("Transfer Receipt", self.receipt_lines(transaction))

    def clear_pending_cheques(self):
        pending = [
            item
            for item in self.transactions
            if item.get("account") == self.account["account_number"]
            and item.get("type") == "Cheque Deposit"
            and item.get("status") == "Pending clearance"
        ]
        if not pending:
            messagebox.showinfo("Cheque Clearance", "There are no pending cheques to
clear.", parent=self.root)
            return

        total = 0.0
        cheque_list = []
        for item in pending:
            amount = float(item["amount"])
            total += amount
            self.account["balance"] += amount
            item["status"] = "Cleared"
            cheque_list.append(item.get("cheque_number", "Unknown"))
            self.record_transaction(
                self.account["account_number"],
                "Cheque Clearance",
                amount,
                self.account["balance"],
                reference=item.get("reference"),
                cheque_number=item.get("cheque_number"),
            )
        if self.save_data():
            self.refresh_menu()
            self.show_receipt(
                "Cheque Clearance Receipt",
                [
                    f"Date: {datetime.now().astimezone().strftime('%d-%m-%Y
%H:%M:%S')}",
                    "Status: Cheques cleared",
                    "Cheque number(s): " + ", ".join(cheque_list),
                    f"Total credited: Rs. {total:.2f}",
                    f"Available balance: Rs. {self.account['balance']:.2f}",
                ],
            )
```

```
    def change_pin(self):
        values = self.ask_fields(
            "Change ATM PIN",
            [("Current PIN", True), ("New 4-digit PIN", True), ("Confirm new PIN",
True)],
        )
        if values is None:
            return
        old_pin, new_pin, confirm = values
        if not self.pin_matches(self.account, old_pin):
            messagebox.showerror("Incorrect PIN", "The current PIN is incorrect.", par-
ent=self.root)
            return
        if len(new_pin) != 4 or not new_pin.isdigit():
            messagebox.showerror("Invalid PIN", "The PIN must contain exactly 4 dig-
its.", parent=self.root)
            return
        if new_pin == old_pin:
            messagebox.showerror("Invalid PIN", "The new PIN must differ from the cur-
rent PIN.", parent=self.root)
            return
        if confirm != new_pin:
            messagebox.showerror("PIN mismatch", "The PINs do not match.", par-
ent=self.root)
            return

        salt = secrets.token_hex(16)
        self.account["pin_salt"] = salt
        self.account["pin_hash"] = self.make_pin_hash(new_pin, salt)
        if self.save_data():
            messagebox.showinfo("PIN changed", "Your ATM PIN was changed successfully.",
parent=self.root)

    def mini_statement(self):
        account_number = self.account["account_number"]
        account_transactions = [
            item for item in self.transactions if item.get("account") == account_number
        ][-10:]
        lines = [
            f"Account: {account_number}",
            f"Available balance: Rs. {self.account['balance']:.2f}",
            f"Pending cheques: Rs. {self.pending_cheque_total(account_number):.2f}",
            "",
            "Recent transactions:",
            "----------------------------------------",
        ]
        if not account_transactions:
            lines.append("No transactions available.")
        else:
            for item in account_transactions:
                lines.extend(
                    [
                        f"{self.display_time(item.get('date'))} | {item.get('type')}",
                        f"Amount: Rs. {float(item.get('amount', 0)):.2f} | "
                        f"Balance: Rs. {float(item.get('balance', 0)):.2f}",
                        f"Status: {item.get('status', 'Completed')} | Ref:
{item.get('reference', 'N/A')}",
                        "----------------------------------------",
                    ]
```

```
                )
        self.show_receipt("Mini Statement", lines)

    def logout(self):
        self.account = None
        self.show_home()


def main():
    root = tk.Tk()
    ATMApp(root)
    root.mainloop()


if __name__ == "__main__":
    main()
```

# 5. Results and Discussion

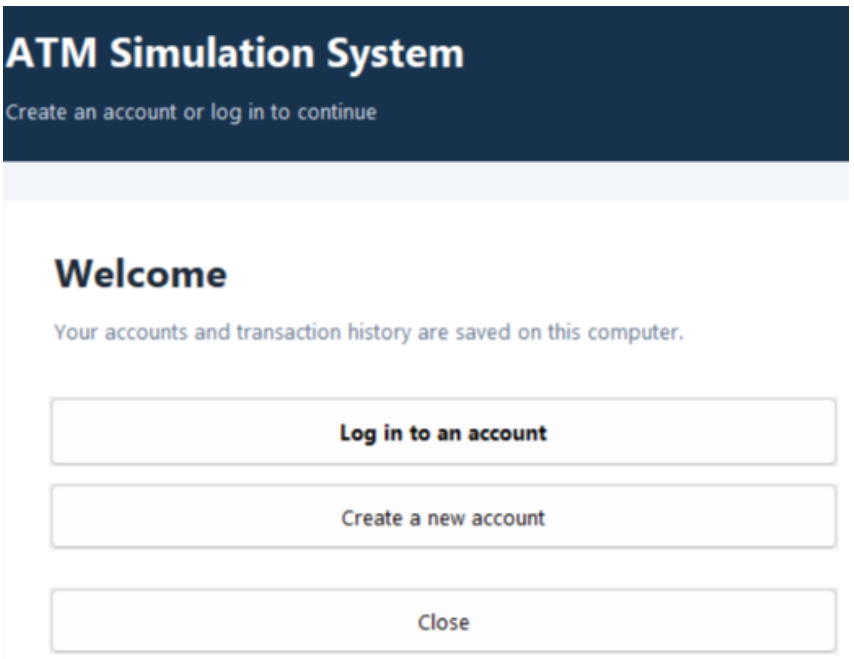



Figure1: Login screen

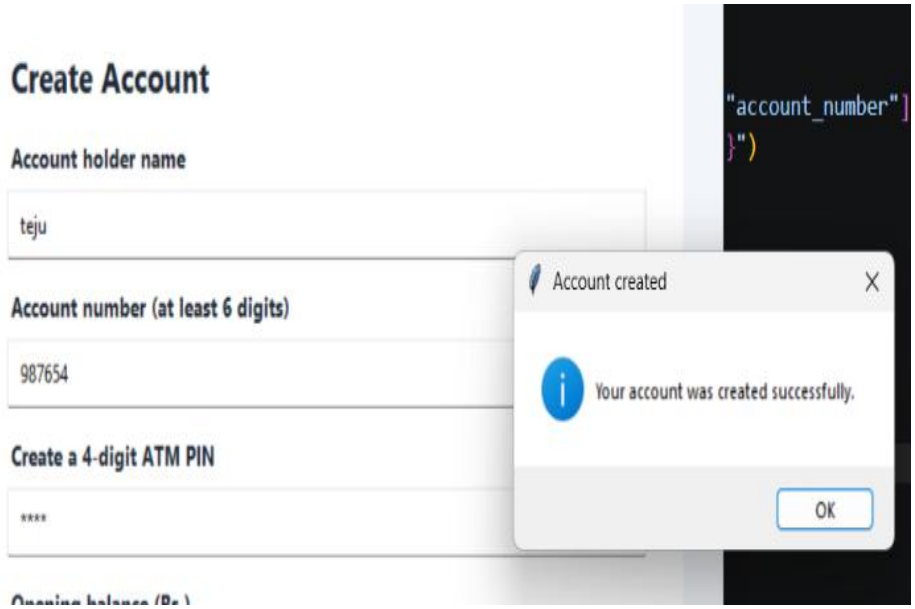



Figure 2: login successful screen

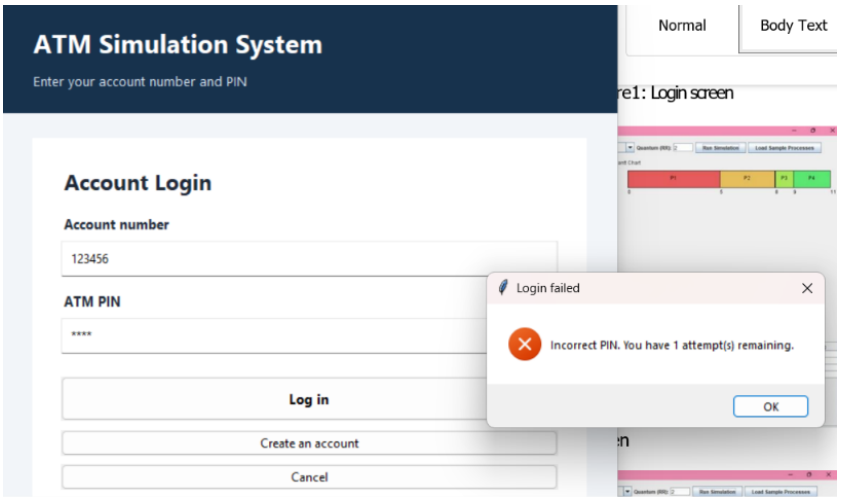



Figure 3: login invalid screen

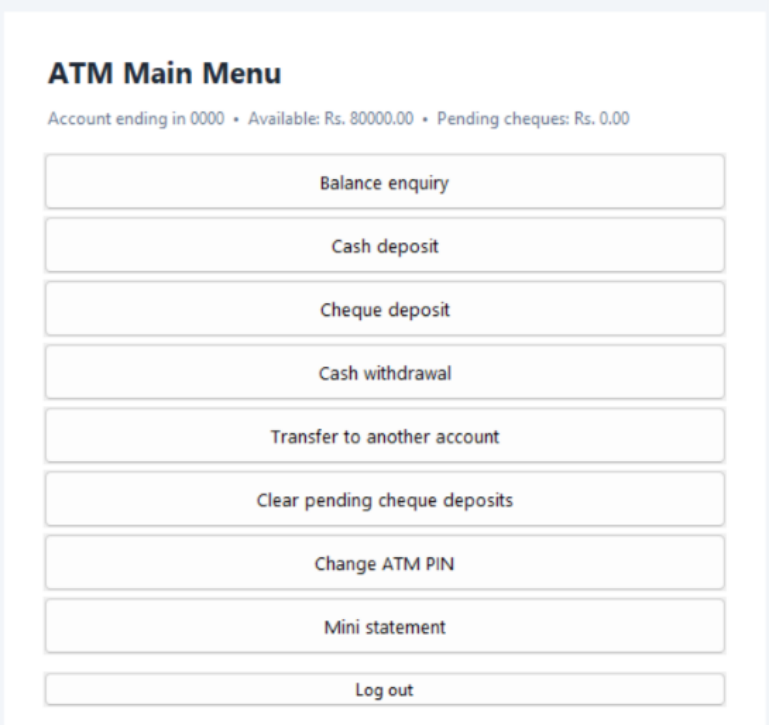



Figure 4:Main menu

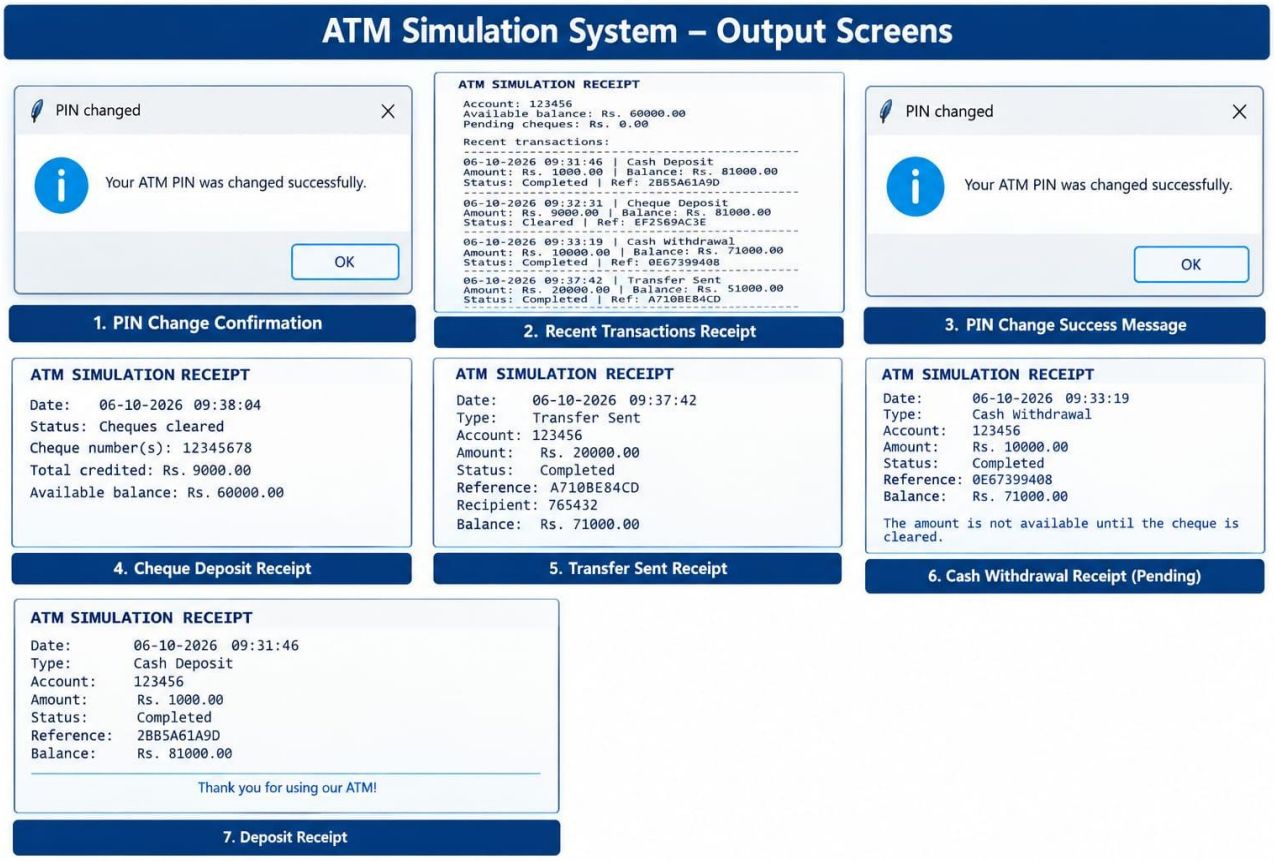



Figure 5: Transactions and user operations

**Summary of Results**

The ATM Simulation System successfully demonstrates the basic operations of an ATM. The system accurately performs user authentication, balance enquiry, cash withdrawal, cash deposit, PIN change, and logout operations. It also updates the account balance after every successful transaction and displays the transaction status to the user.

Interpretation

The simulation provides a simple and user-friendly way to understand ATM operations. Authentication improves security by allowing only valid users to access the account. Validation prevents incorrect transactions such as withdrawing more money than the available balance, while error handling provides suitable messages for invalid inputs.

Achievement of Objectives

The project successfully met its objectives by implementing the major ATM functions, including PIN authentication, balance checking, withdrawal, deposit, PIN management, database handling, and transaction processing. The system also provides a clear interface for users to perform banking operations and view the results.

Implications

# 6. Mapping POs and SDGs

## 6.1 Program Outcomes (POs) Mapping

Table 1: Mapping of Program Outcomes (POs) to Project Relevance

| PO No. | Program Outcome | Relevance |
|---|---|---|
| **PO1** | Engineering Knowledge | Applied programming, database, and software engineering concepts to develop a functional ATM Simulation System. |
| **PO2** | Problem Analysis | Analyzed common ATM operations such as authentication, withdrawal, deposit, balance enquiry, and PIN management to develop suitable solutions. |
| **PO3** | Design/Development of Solutions | Designed and developed an ATM system that accepts user input, validates transactions, processes requests, and displays appropriate results. |
| **PO4** | Conduct Investigations of Complex Problems | Tested different transaction scenarios such as valid PINs, incorrect PINs, insufficient balance, deposits, and withdrawals to verify system reliability. |
| **PO5** | Engineering Tool Usage | Utilized programming languages, database technologies, GUI tools, and development environments to implement the ATM simulation and manage account information. |
| **PO8** | Individual and Collaborative Team Work | Worked independently on coding and documentation while collaborating with team members for testing, reviewing, and improving the system. |
| **PO9** | Communication | Presented the system workflow, interface, transaction results, and project documentation clearly through demonstrations and reports. |
| **PO11** | Life-Long Learning | Explored ATM transaction processing, database connectivity, authentication techniques, and software development concepts through self-learning and practical implementation. |

Table 1: Mapping of Program Outcomes (POs) to Project Relevance

## 6.2 Sustainable Development Goals (SDGs) Mapping

| SDG No. | Goal | Relevance |
|---|---|---|

| | | |
|---|---|---|
| **SDG 4** | Quality Education | The ATM Simulation System provides practical learning of programming, databases, authentication, and transaction processing, making software concepts more interactive and understandable. |
| **SDG 8** | Decent Work and Economic Growth | Develops programming, problem-solving, database management, and software development skills that are useful for employment in IT and banking technology sectors. |
| **SDG 9** | Industry, Innovation and Infrastructure | Demonstrates the use of software and database technologies to simulate a real-world banking service and encourages innovation in digital financial systems. |
| **SDG 10** | Reduced Inequalities | A digital ATM simulation demonstrates how technology can support accessible and convenient banking services for different users. |
| **SDG 12** | Responsible Consumption and Production | A software-based ATM simulation reduces the need for physical resources during training and provides a reusable platform for learning banking operations. |

Table 2: Mapping of Sustainable Development Goals (SDGs) to Project Relevance

### 6.3 . Program Specific Outcomes (PSOs) Mapping

| No. | Program Specific Outcome | Relevance |
|---|---|---|
| **PSO1** | Apply software design and development practices in emerging areas such as Cloud Computing, High-Performance Computing, Data Analytics and Cyber Security. | The ATM Simulation System applies structured software design, programming, database management, authentication, transaction processing, and security practices to develop a practical banking application. |
| **PSO2** | Employ modern computer languages, environments and platforms for entrepreneurship, innovative careers and higher studies. | The project uses modern programming languages and development environments to build a practical ATM simulation, strengthening programming, problem-solving, and application development skills for careers and higher studies. |

Table 3: Mapping Program Specific outcomes(PSOs) to Project Relavance

# 7. Conclusion

This project successfully implemented a Java-based CPU Scheduling Simulator capable of demonstrating fundamental scheduling algorithms including FCFS, SJF, Priority Scheduling, and Round-Robin. Through an interactive GUI, the system allows users to input process data, visualize execution through Gantt charts, and observe key performance metrics such as waiting time and turnaround time. The simulator effectively bridges theory and practice, helping learners understand how operating systems allocate CPU resources and optimize process execution. By comparing algorithm performance, this project highlights how scheduler selection impacts system efficiency under different workloads.

For future enhancements, the simulator can be extended to support preemptive variations, multi- level queue scheduling, real-time scheduling models, and dynamic priority handling. Additional features such as CPU burst prediction using machine learning, memory and I/O scheduling integration, and real-time graph-based analytics could further enrich understanding. Cloud- based deployment and mobile application support may also improve accessibility. Continuing development in these areas will strengthen the tool's educational value and expand its applicability in academic and research environments.

# Bibliography

## Online Resources


1. Python Software Foundation – Built-in Functions (input, print, float, len). https://docs.python.org/3/library/functions.html

2. Python Software Foundation – Built-in Types (str, list, dict methods). https://docs.python.org/3/library/stdtypes.html

3. Python Software Foundation – The Python Tutorial: Errors and Exceptions. https://docs.python.org/3/tutorial/errors.html

4. Python Software Foundation – datetime: Basic date and time types. https://docs.python.org/3/library/datetime.html

5. Python Enhancement Proposal 8 – Style Guide for Python Code. https://peps.python.org/pep-0008/

6. OWASP – Authentication Cheat Sheet. https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html


## Research article references


[1] Anderson, R. (2020). Security Engineering: A Guide to Building Dependable Distributed Systems (3rd ed.). Wiley.

[2] Shneiderman, B., Plaisant, C., Cohen, M., Jacobs, S., & Elmqvist, N. (2016). Designing the User Interface: Strategies for Effective Human-Computer Interaction (6th ed.). Pearson.

[3] Lutz, M. (2013). Learning Python (5th ed.). O'Reilly Media.

[4] Downey, A. B. (2015). Think Python: How to Think Like a Computer Scientist (2nd ed.). O'Reilly Media.

[5] Sommerville, I. (2016). Software Engineering (10th ed.). Pearson Education.

[6] Matthes, E. (2023). Python Crash Course (3rd ed.). No Starch Press.